

Recap — Assignment #6: The location map

Recap — Assignment #6: The location map

Branch: `feature/location-map` **Trello cards:** Person B, Sprint 2 — *Upgrade location map, Locked / unlocked locations, Show current player location* **Date:** 2026-08-19

Assignment #5 gave the player a view of what they are **carrying**. This one gives them a view of where they can **go** — and, for the first time, where they can no longer go.

Part 0 — What we built

The old map was a flat list of buttons with three states: current, reachable, not reachable. It worked out "reachable" from one thing only — `connectedLocationIds`.

Meanwhile Person C had added two systems the map had never heard of. The map was quietly lying to the player about both.

Now it shows four states, plus a locked look for places you have never been, plus a header telling you where you are standing and how much of the city you have seen.

Part 1 — The two things the map did not know

It did not know roads can be destroyed

Person C's timed events can cut a road permanently. In Pompeii, at minute 140, the `roof_collapse` event fires:

```

{
  id: "roof_collapse",
  triggerTime: 140,
  healthChange: -25,
  blockedRoutes: [
    { fromLocationId: "forum", toLocationId: "baths" },
    { fromLocationId: "temple", toLocationId: "villa" },
  ],
}

```

The server enforces it and refuses the move with `ROUTE_BLOCKED` — `server/services/gameActionService.js:80`.

The old map still drew Forum→Baths as a normal, clickable road. The player clicked it, and the move simply failed. The screen was showing a road the game no longer allowed.

It did not know where you had already been

`game.discoveredLocationIds` records every location you have set foot in. It starts with the scenario's start location and grows on each successful move.

The map ignored it entirely. The Harbour looked exactly as familiar as the Forum you were standing in, from the first second of the game.

Part 2 — Four states instead of three

State	What it means	Clickable?
You are here	your current location	no
Reachable	connected, and no triggered event has cut the road	yes
Road destroyed	connected, but a triggered event blocked it	no
Out of reach	not connected from where you stand	no

On top of that, any location not in `discoveredLocationIds` is drawn dimmed, with a dashed border, and labelled "**Unexplored**".

Two things are being expressed at once, and it is worth seeing that they are independent:

- **The state** answers *can I walk there right now?* It changes every time you move.

- **Discovered** answers *have I ever been there?* Once true, it never goes back to false.

A location can be reachable and unexplored (the next new room), or explored and out of reach (somewhere you visited an hour ago, now across the city). Keeping them as separate ideas — a `state` string and an `isDiscovered` boolean — is what lets the map show both without an explosion of special cases.

Part 3 — Where the logic lives, and why not in the component

None of the state-working-out happens inside `LocationMap.jsx`. It happens in a new file, `client/src/utils/mapState.js`, which exports plain functions that take data and return data.

```
const rows = describeLocations({
  locations, events, currentLocationId, triggeredEventIds,
  discoveredLocationIds,
});
```

Each row that comes back already carries everything the map needs:

```
{ location, state: "blocked", label: "Road destroyed", isDiscovered:
  false, canMove: false }
```

The component then does nothing but draw. No conditionals about eruptions, no set arithmetic, just a `.map()` over rows.

Why split it out? Because a **pure function** — one that only reads its arguments and only returns a value, touching nothing else — can be run and checked without a browser, without React, without a server. That is exactly what was done below, 2048 times.

Logic buried inside a component can only be tested by rendering the component. Logic in a plain function can be tested by calling it.

Part 4 — The duplication, said out loud

`mapState.js` contains a copy of the server's `isRouteBlocked`. The same rule now exists in two places, in two languages of the same language.

This is deliberate, and it is a real trade-off, so know what you are buying.

Why copy it: without it the map cannot grey out a dead road, and the player learns the road is gone only by clicking it and getting an error. The map would be

knowingly showing a lie.

What it costs: if someone changes the blocking rule on the server and forgets the client, the two drift apart and the map starts lying again — quietly this time.

What keeps it safe:

1. **The server is the authority.** The client copy only decides whether a button looks clickable. If it is wrong and the player somehow fires the move, the server still refuses with `ROUTE_BLOCKED` and the error now appears next to the map. A wrong client causes a cosmetic bug, never an illegal move.
2. **The comment at the top of the function says so**, and names the server file to keep in sync.
3. **The parity check below** proves they agree today.

The general name for this is a **client-side guard**. The rule of thumb: guards may duplicate server rules for the sake of the interface, but the server must never trust them.

Part 5 — Proving the copy is honest

Rather than trusting that the copy was faithful, both functions were run against each other over **every combination**: all 2^5 subsets of Pompeii's five events $\times$ all 8×8 ordered location pairs.

```
combinations checked: 2048  
mismatches: 0
```

That is exhaustive, not a sample — for this scenario there is no input left to try.

Both check scripts live in the scratchpad, not the repo. **Worth considering**: the parity check would be more valuable as a real test in `server/tests/`, so it runs in CI and fails the day someone edits one side. It was left out because the client has no test runner at all — no vitest, no jest. That gap is now the main reason frontend work here can only be checked by eye.

Part 6 — Errors move next to the thing they are about

Assignment #5 established the pattern: an error appears on the card whose button you clicked. The map now joins it.

```
const moveError = failedActionType === "MOVE" ? gameStore.error :  
  null;
```

A refused move — `ROUTE_BLOCKED`, `INVALID_MOVE`, `ALREADY_AT_LOCATION` — now prints inside the map panel. The scene panel keeps everything else:

```
{gameStore.error && !itemError && !moveError ? ( ... ) : null}
```

This is the payoff from the `failedAction` field added in #5. Without it there would be no way to ask "was this a movement error?" — the codes alone do not say.

Notice the shape of that condition: the scene panel is now defined by exclusion — *show anything nobody else claimed*. That works, but it grows a new `&& !x` every time a panel claims a category. Two is fine. At four it would be worth inverting: let each panel declare what it owns, and give the scene whatever is left over.

Part 7 — Words to remember

Word	What it means
Pure function	Reads only its arguments, returns a value, changes nothing else
Client-side guard	A copy of a server rule, used only to shape the interface
Authority	The one place whose answer counts — here, always the server
Exhaustive check	Trying every possible input, not a sample of them
Derived state	A value worked out from other data rather than stored

`state`, `label` and `canMove` are all derived state. Nothing stores "the Baths are blocked" — it is recomputed from the triggered events every render. Derived state cannot go stale, which is why it is preferred to storing a copy.

Part 8 — Files

New

File	What it does
<code>client/src/utils/mapState.js</code>	The four states, the discovered flag, and the copy of the server's blocked-route rule

Changed

File	What changed
<code>client/src/components/game/LocationMap.jsx</code>	Rewritten to draw rows from <code>describeLocations</code> ; added the "You are at" header and explored count
<code>client/src/components/game/LocationMap.css</code>	Styles for the blocked state (red) and unexplored state (dimmed, dashed)
<code>client/src/pages/games/GamePage.jsx</code>	Passes events, triggered events and discovered locations; routes move errors to the map

Part 9 — What comes next

Sprint 2 stands at **9 of 11** once this merges. Two left, and they go together:

- **Cloudinary setup**
- **Scenario media upload**

Both are a different kind of work from everything so far — a third-party service, an API key, and file uploads. `server/.env.example` already has the three empty Cloudinary variables waiting.

After that, Sprint 3: the admin scenario pages. That will be the first code in the project to use `server/middleware/authorize.js`, which Person A wrote and nobody has ever called.

Checks

Check	Result
<code>npm test</code> (server)	89 pass, 0 fail
<code>npm run lint</code> (client)	Clean, no warnings
<code>npm run build</code> (client)	Succeeded, 77 modules
Map states at the Forum, no events	forum=current, market/baths/temple=reachable, rest=out-of-reach
Map states after <code>roof_collapse</code>	baths flips reachable → blocked , and <code>canMove</code> is false
Discovered flag	visited=true, unvisited=false
Client vs server blocked-route parity	2048 combinations, 0 mismatches

Not checked: how it looks. As in #5, no screenshot was taken — the Chrome extension dropped during the previous assignment and was not brought back. The logic is verified hard; the pixels are not. The dimmed unexplored style and the red blocked style have never been seen. Worth one look before the demo.